

MedROAD V3

Installation and Operation Manual

Pierre Boulanger University of Alberta

October 2026

Contents

Installation and Operation	1
1. Prerequisites	1
2. Install	2
3. Configure	2
4. Obtain MIMIC-IV	2
5. Extract, train, reproduce	3
6. OpenEMR: installation and connection to MedROAD	4
7. Narrative generation (LLM layer)	8
8. Troubleshooting	9
9. Reproducibility notes	11
10. Publishing to GitHub	11
11. Getting help	12

Installation and Operation

Covers installing MedROAD V3, obtaining and preparing MIMIC-IV, training, and reproducing every result in the accompanying paper. The troubleshooting section at the end records failures encountered during real deployment rather than hypothetical ones, and is worth reading before you hit them.

1. Prerequisites

	Requirement	Notes
Python	3.11 or 3.12	On Windows install from python.org, not the Microsoft Store (see 8.2)
Disk	~20 GB	7 GB MIMIC-IV extract, ~2 GB window matrix, models
RAM	16 GB	Training on the full cohort peaks near 10 GB
GPU	optional	CUDA roughly halves training time; CPU works
Docker	optional	Only for the OpenEMR/Kafka stack and transport benchmarks; install per 6.1
PostgreSQL	optional	Only if extracting via SQL rather than the pandas path

MIMIC-IV requires PhysioNet credentialing: complete the CITI training and sign the data use agreement at <https://physionet.org/settings/credentialing/>. Approval typically takes several days, so start this first.

2. Install

```
git clone https://github.com/pboulanger/medroad-v3.git
cd medroad-v3
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install -e ".[all]"
```

Install only what you need if the full set is unnecessary:

Extra	Pulls in	Needed for
(base)	numpy, scipy, scikit-learn, pandas	ETL, calibration, RPM mapping
[ml]	xgboost, shap, torch	training, inference, all experiments
[stream]	kafka-python-ng, faust-streaming	the streaming pipeline
[serve]	fastapi, uvicorn	the FHIR webhook receiver
[llm]	anthropic	narrative generation
[dev]	pytest, ruff	contributing

Verify:

```
pytest -q          # full suite, no infrastructure required
python -c "import medroad_v3; print(medroad_v3.config.N_FEATURES)" # 86
```

3. Configure

```
cp .env.example .env
```

Edit `.env`. Only `MODEL_DIR` matters for training and the offline experiments; the rest are needed for the live stack.

```
OPENEMR_BASE_URL=http://localhost:8300
OPENEMR_CLIENT_ID=          # from section 6
OPENEMR_CLIENT_SECRET=
WEBHOOK_SECRET=change-me    # HMAC shared secret
KAFKA_BOOTSTRAP=localhost:9092
MODEL_DIR=./models_saved
RISK_THRESHOLD=0.72         # re-derive per site, see paper Sec. 6.4
ANTHROPIC_API_KEY=          # narrative generation only
```

Never commit `.env`. It is gitignored.

4. Obtain MIMIC-IV

Download only the seven tables used, roughly 6 GB compressed rather than the full dataset. Do **not** place it inside OneDrive, Dropbox or iCloud (see 8.1).

```
mkdir -p ~/mimic/full && cd ~/mimic/full
BASE=https://physionet.org/files/mimiciv/3.1
for f in hosp/admissions.csv.gz hosp/patients.csv.gz \
        hosp/labevents.csv.gz hosp/prescriptions.csv.gz \
        icu/icustays.csv.gz icu/chartevents.csv.gz icu/d_items.csv.gz; do
    curl -u YOUR_USERNAME -C - --create-dirs -o "$f" "$BASE/$f"
done
```

-C - resumes rather than restarting, which matters for the 3 GB chartevents.csv.gz. **Verify integrity before extracting**, because a truncated gzip fails deep inside the ETL and wastes an hour:

```
python - <<'EOF'
import gzip, pathlib, sys
bad = []
for p in sorted(pathlib.Path("~/mimic/full").expanduser().rglob("*.gz")):
    try:
        with gzip.open(p, "rb") as fh:
            while fh.read(1 << 20):
                pass
        print("OK", p.name)
    except Exception as e:
        bad.append(p); print("DAMAGED", p.name, type(e).__name__)
sys.exit(1 if bad else 0)
EOF
```

5. Extract, train, reproduce

5.1 Build the extract

Two routes produce the same four CSVs. Use pandas unless you already run PostgreSQL with MIMIC-IV loaded.

```
# pandas, no database
python -m medroad_v3.training.extract_raw \
    --raw-dir ~/mimic/full --out-dir ./mimic_extract --blank-hours 6

# or SQL
psql -d mimiciv -f sql/extract_mimic.sql
```

Expect roughly 25,000 cardiac ICU stays with an event rate near 28%. A rate above 35% means the label is capturing routine care; the extractor warns and --blank-hours is the first thing to raise.

The outcome is a composite of vasopressor initiation, invasive ventilation and ICU death, blanked for six hours after admission. Section 5 of the paper states the definition and its limitations; read it before changing anything here.

5.2 Train

```
# verify the pipeline on 50 stays first, ~2 minutes
python scripts/retrain.py --mimic-dir ./mimic_extract --quick --lenient

# a scaled pilot
python scripts/retrain.py --mimic-dir ./mimic_extract --max-stays 5000

# the full cohort
python scripts/retrain.py --mimic-dir ./mimic_extract
```

`retrain.py` gates between stages because every failure that matters here is silent: a wrong itemid mapping, a mislabelled cohort, a constant sequence tensor and an out-of-range feature all train cleanly and produce plausible metrics. Any gate failing stops the run and writes `results/retrain_report.json`.

Windows users may prefer `.\scripts\retrain.ps1 -MimicDir .\mimic_extract`, which handles interpreter resolution and quotes paths containing spaces.

5.3 Reproduce the paper’s results

```
python -m medroad_v3.experiments.ablation          --data mimic_windows.csv
python -m medroad_v3.experiments.rpm_degradation   --data mimic_windows.csv
python -m medroad_v3.experiments.baselines         --data mimic_windows.csv
python -m medroad_v3.experiments.latency           --data mimic_windows.csv
python -m medroad_v3.experiments.latency_scaling   --data mimic_windows.csv
```

Each writes a JSON result and a LaTeX table or figure under `results/`, ready to paste. `baselines` requires `models_saved/split.json`, written during training, so that it scores the patients the models were actually held out from.

All experiments accept `--synthetic` to exercise the pipeline without credentialed data. Synthetic figures verify plumbing and must never be reported.

6. OpenEMR: installation and connection to MedROAD

Required for end-to-end operation and the transport latency benchmark. Not required for training or for any of the offline experiments.

MedROAD connects to OpenEMR in both directions. It subscribes to FHIR Subscriptions so that new Observations are pushed to its webhook, and it writes alerts back as Flag, CommunicationRequest, DocumentReference and ServiceRequest resources. Both directions authenticate through OAuth2, so the client registration in 6.3 is the step that matters most.

6.1 Install Docker

OpenEMR is distributed as a container. Install Docker first if you have not.

Windows. Enable WSL 2 from an elevated PowerShell, reboot, then install Docker Desktop:

```
wsl --install
# reboot, then install from
# https://docs.docker.com/desktop/setup/install/windows-install/
```

Check Task Manager → Performance → CPU shows **Virtualization: Enabled**. If it says Disabled, enable Intel VT-x or AMD-V in your BIOS first. Launch Docker Desktop and wait for the whale icon in the tray to stop animating, 30 to 60 seconds.

macOS. Install Docker Desktop from <https://docs.docker.com/desktop/setup/install/mac-install/>, choosing the Apple silicon or Intel build to match.

Linux. Install Docker Engine and the compose plugin from your distribution, or follow <https://docs.docker.com/engine/install/>.

Verify before going further:

```
docker --version
docker compose version
docker info --format "{{.ServerVersion}}"      # fails if the daemon is down
docker run --rm hello-world
```

Docker Desktop is free for personal use, education and non-commercial open source. Organisations above 250 employees or \$10M revenue need a paid subscription; academic research use is free.

6.2 Install OpenEMR

Two options. Use the first unless OpenEMR needs to live apart from the pipeline.

Option A: as part of the MedROAD stack. Brings up OpenEMR, MariaDB, Zookeeper and Kafka together.

```
cp .env.example .env      # compose reads this; it must exist
docker compose up -d mariadb openemr zookeeper kafka
docker compose logs -f openemr
```

Option B: OpenEMR alone. No Kafka, no MedROAD containers. Useful for development, or when OpenEMR runs on a different host.

```
cp .env.example .env
docker compose -f docker/openemr-standalone.yml up -d
docker compose -f docker/openemr-standalone.yml logs -f openemr
```

Either way, wait for OpenEMR has been successfully installed in the logs. First boot installs the database schema and takes three to five minutes; the healthcheck allows four before reporting unhealthy, so follow the logs rather than the health status.

Both compose files set `MYSQL_ROOT_HOST` to `%`. OpenEMR connects to MariaDB from a different container address, and MariaDB restricts root to localhost by default, so without this the installer fails with “unable to connect to database as root”. If you have already started the stack once without it, the setting is baked into the volume and you must `down -v` before it takes effect.

Log in at <http://localhost:8300> with `admin / Admin1234!`. Change the password before any non-local use.

Confirm the server is up and the FHIR API is enabled:

```
curl -s http://localhost:8300/apis/default/fhir/metadata | head -c 200
```

A FHIR CapabilityStatement means both are working. An HTML login page means the API is off; see 6.3.

Useful commands.

```
docker compose stop      # stop, keep data
docker compose start     # resume
docker compose down      # stop and remove containers, keep volumes
docker compose down -v   # also delete the database: full reset
docker compose logs openemr --tail 50
docker compose exec openemr bash    # shell inside the container
```

6.3 Verify the APIs are enabled

The compose file turns on the REST and FHIR APIs at install time through `OPENEMR_SETTING_*` variables. If you installed OpenEMR another way, enable them manually:

1. Log in at <http://localhost:8300> as `admin / Admin1234!`

2. Administration → Config → Connectors
3. Enable **Enable OpenEMR REST API** and **Enable FHIR REST API**
4. Enable **Enable OAuth2 Password Grant** (MedROAD uses the resource owner password credentials flow for unattended service authentication)
5. Save

Menu wording varies between OpenEMR 7.x releases; the settings are grouped under Connectors in all of them.

6.4 Register the OAuth2 client

Registration is self-service, but a newly registered client is **disabled by default** and will return `invalid_client` until an administrator enables it. This is the step most installations miss.

1. Open <http://localhost:8300/interface/smart/register-app.php>
2. Register with:

Field	Value
Application name	MedROAD V3
Redirect URI	http://localhost:8001/callback
Launch URI	http://localhost:8001/launch
Client type	Confidential
Scopes	see 6.5

3. **Copy the `client_id` and `client_secret` immediately.** The secret is shown once and cannot be retrieved afterwards; if you lose it, delete the client and register again.
4. Go to Administration → System → API Clients, find **MedROAD V3**, and **enable** it. Grant the registered scopes here as well.

6.5 Scopes

MedROAD needs read access to the resources it monitors and write access to the four it emits. Request these at registration:

```
openid offline_access
system/Observation.read      system/Patient.read
system/Encounter.read        system/MedicationRequest.read
system/Flag.write            system/CommunicationRequest.write
system/DocumentReference.write system/ServiceRequest.write
```

`system/` scopes are correct here rather than `user/` or `patient/`: MedROAD runs unattended as a backend service, not on behalf of a logged-in clinician. Requesting `user/` scopes will appear to work during registration and then fail at write-back.

6.6 Wire the credentials into MedROAD

```
OPENEMR_BASE_URL=http://localhost:8300
OPENEMR_CLIENT_ID=<client_id from 6.4>
OPENEMR_CLIENT_SECRET=<client_secret from 6.4>
OPENEMR_USERNAME=admin
OPENEMR_PASSWORD=Admin1234!
```

Verify the credentials independently of MedROAD before going further:

```
curl -s -X POST http://localhost:8300/oauth2/default/token \
  -d grant_type=password \
  -d client_id="$OPENEMR_CLIENT_ID" \
  -d client_secret="$OPENEMR_CLIENT_SECRET" \
  -d username=admin -d password='Admin1234!' \
  -d scope='openid offline_access system/Observation.read' | head -c 300
```

An `access_token` means registration is complete. `invalid_client` almost always means the client was never enabled in 6.3 step 4.

6.7 Register the FHIR Subscriptions

With credentials in place, have MedROAD create its Subscriptions. Each is a REST-hook pointing at the webhook receiver, filtered by LOINC category.

```
python -m medroad_v3.main setup          # creates the Subscriptions
python -m medroad_v3.main teardown       # removes them again
```

Inside Docker the webhook is reachable at `http://webhook:8001/fhir/webhook`; from the host it is `http://host.docker.internal:8001/fhir/webhook`. Set `WEBHOOK_URL` accordingly, because OpenEMR resolves this address from inside its own container and `localhost` there refers to OpenEMR itself.

6.8 Run and verify end to end

```
python -m medroad_v3.main webhook        # terminal 1
python -m medroad_v3.main stream         # terminal 2
python -m medroad_v3.main infer-test     # terminal 3: synthetic end-to-end check
```

To exercise the real path, post an Observation for a test patient and watch it arrive:

```
curl -s -X POST http://localhost:8300/apis/default/fhir/Observation \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/fhir+json" \
  -d '{"resourceType": "Observation", "status": "final",
    "category": [{"coding": [{"code": "vital-signs"}]}],
    "code": {"coding": [{"system": "http://loinc.org", "code": "8867-4"}]},
    "subject": {"reference": "Patient/1"},
    "effectiveDateTime": "2026-01-01T12:00:00Z",
    "valueQuantity": {"value": 110, "unit": "/min"}}'
```

The webhook log should show the delivery, the stream log should show the window updating, and a score above threshold should produce a Flag on the patient chart.

6.9 OpenEMR troubleshooting

unable to connect to database as root. MariaDB restricts root to localhost by default. The compose file sets `MYSQL_ROOT_HOST` to `%`. If you changed it after the first start, the setting is already baked into the volume: `docker compose down -v` and start again.

Container reports unhealthy during first boot. Normal for the first three to five minutes. Follow `docker compose logs -f openemr` rather than the healthcheck.

/apis/default/fhir/metadata returns a login page. The FHIR API is disabled; see 6.3.

invalid_client on the token request. The registered client was never enabled by an administrator; see 6.4 step 4.

Write-back fails while reads succeed. The client holds `user/` rather than `system/` scopes; see 6.5.

Subscriptions register but no events arrive. `WEBHOOK_URL` is set to an address OpenEMR cannot resolve from inside its container. Use `host.docker.internal` from the host, or the service name within compose.

Starting over. `docker compose down -v` removes the volumes along with the database, which is the only reliable way to re-run the installer.

7. Narrative generation (LLM layer)

Optional. The pipeline runs, trains and alerts without it; narrative generation adds a three-sentence clinical summary to alerts that cross the threshold. When no API key is configured the system uses the deterministic fallback template and degrades to a slightly less fluent alert rather than to no alert.

7.1 Before enabling it: data governance

Read this before pointing the narrative layer at any real data.

The narrative call sends patient-derived values to a third-party API. MedROAD constructs the prompt exclusively from numeric values, LOINC codes and SHAP magnitudes, and never includes free-text EHR content, which forecloses prompt injection and limits what leaves the deployment. It does not make the data non-identifiable.

Two separate obligations apply.

For MIMIC-IV: the data is de-identified but credentialed, and the data use agreement restricts sharing with third parties. Whether API calls constitute sharing is a question for PhysioNet’s current guidance rather than for this manual. Check it before running narrative generation on MIMIC-IV, and default to the fallback template for published benchmarks.

For clinical deployment: sending patient data to an external service engages HIPAA in the United States, PIPEDA and provincial health information acts in Canada, and GDPR in Europe. A business associate agreement or equivalent is normally required. This is an institutional decision, not a configuration choice.

The system is designed so that disabling the LLM costs you fluency and nothing else. If governance is unresolved, leave `ANTHROPIC_API_KEY` empty and ship.

7.2 Obtain and configure a key

1. Create an account at <https://console.anthropic.com>
2. Generate an API key under API Keys
3. Add it to `.env`:

```
ANTHROPIC_API_KEY=sk-ant-...
CLAUDE_MODEL=claude-sonnet-4-6      # default
CLAUDE_MAX_TOKENS=300               # a three-sentence narrative needs ~150
CLAUDE_TEMPERATURE=0.2              # low: summaries should be reproducible
```

The key is read once at construction. `.env` is gitignored; keep it that way.

7.3 What the prompt contains

The system prompt is fixed at deployment and encodes the output contract: three sentences covering the clinical picture, the SHAP-identified drivers, and a recommended action; cardiologist-level register; a mandatory provenance string marking the text as AI-generated and requiring physician review; and a prohibition on definitive diagnostic phrasing.

The user prompt is assembled per alert from the windowed vital statistics, the top five SHAP attributions, recent laboratory values, the ensemble score with its components, and the active medication list. Every field is numeric or drawn from a controlled vocabulary.

Sampling temperature is 0.2, well below default, trading variety for consistency.

7.4 Test it

```
python -m medroad_v3.main infer-test      # exercises the full path incl. narrative
```

With a key configured you should see a generated three-sentence narrative. With the key absent you should see the fallback template, which is the correct behaviour and worth confirming deliberately.

7.5 Cost and rate limits

Narrative generation is conditional on $R \geq \tau$, so cost scales with alert rate rather than window count. Each call sends roughly 400 to 600 input tokens and returns under 200.

Estimate before enabling it on a live unit: multiply your measured alert rate by the per-call token cost at current pricing. A unit alerting a few times per patient per shift costs little; one alerting on every window because the threshold was never re-derived locally (see paper Section 6.4) costs a great deal. Derive the threshold first.

The API also rate-limits. On a burst of simultaneous alerts the generator catches the error and falls back to the template rather than dropping the alert.

7.6 Troubleshooting

No narrative appears, alerts otherwise work. Expected when `ANTHROPIC_API_KEY` is unset. Confirm with `python -c "from medroad_v3 import config; print(bool(config.ANTHROPIC_API_KEY))"`.

`anthropic.AuthenticationError`. The key is wrong, revoked, or the `.env` was not loaded. Check that `.env` sits in the working directory.

`anthropic.RateLimitError` in logs with alerts still firing. Working as intended: the fallback absorbed it. Investigate only if it is frequent, which suggests the threshold is too permissive.

Narratives rejected and replaced by the template. The factual grounding check found claims not traceable to prompt fields. Occasional rejection is the safety mechanism working; persistent rejection suggests a model or prompt change and warrants inspecting the raw outputs.

`ModuleNotFoundError: anthropic`. Install the extra: `pip install -e ".[llm]"`.

8. Troubleshooting

8.1 OSError: [Errno 22] or “cloud file provider exited unexpectedly”

The dataset is on OneDrive, Dropbox or iCloud and the files are placeholders: present in the listing at full size, contents not local. Reading one fails deep inside the decompressor.

Move the data off cloud-synced storage. Forcing a download works temporarily but the files get evicted again:

```
Get-ChildItem -Recurse .\mimic_subset -File |
  Where-Object { $_.Attributes -match 'Offline' } | Measure-Object
robocopy ".\mimic_subset" "C:\mimic\mimic_subset" /E /MOVE /R:3 /W:5
```

Syncing credentialed PhysioNet data to third-party cloud storage may also breach the data use agreement.

8.2 Microsoft Store Python

The Store build runs sandboxed with redirected filesystem writes and cannot trigger OneDrive hydration. It has also produced interpreter-level failures inside pandas during long runs. Install from `python.org` instead. On Windows use `python`, never `python3`, which resolves to the Store stub.

8.3 `ModuleNotFoundError: No module named 'kafka.vendor.six.moves'`

`kafka-python` 2.0.2 does not import on Python 3.12. The dependency spec already pins the maintained fork, which keeps the same import name:

```
pip install "kafka-python-ng>=2.2.2"
```

8.4 `EOFError: Compressed file ended before the end-of-stream marker`

A truncated download. Re-fetch with `curl -C -` to resume. Everything before the truncation point is recoverable if re-downloading is impractical, but a partial cohort is not suitable for reported results.

8.5 `UnicodeDecodeError: 'charmap' codec can't decode byte`

Python on Windows defaults to the system code page, usually cp1252, and fails on any file containing a dash, arrow or accented character. Every text read in this project states `encoding="utf-8"` for that reason. If you see this in your own additions, pass the encoding explicitly; setting `PYTHONUTF8=1` works as a global workaround.

8.6 failed to connect to the docker API

Docker Desktop is not running. Start it, wait for the tray icon to go steady, then confirm with `docker info`. Installation is covered in 6.1. If it stays unresponsive, `wsl --shutdown` followed by a restart usually clears a wedged WSL2 backend.

OpenEMR-specific problems are covered in section 6.9; narrative generation in section 7.6.

8.7 Input and parameter tensors are not at the same device

Fixed in current versions; `load_model` places the module on the requested device. If you see it, your working copy predates that fix.

8.8 Gate failure: “feature column(s) fall outside [-1, 1]”

Every engineered feature is bounded by construction. Out-of-range values indicate a defect, most likely a clock misalignment in the ETL or a dimension count that moved without its divisor. Do not disable the gate: tree models tolerate the corruption silently while neural models are destroyed by it, which produces a confident and wrong architectural conclusion.

8.9 Gate failure: “sequences are constant”

`build_sequences` was called without `patient_ids` and `window_starts`, so the LSTM and Transformer receive identical copies of one window and score exactly 0.5. Pass both.

8.10 Training aborts: “patients appear on both sides of the split”

Grouping failed. Splits must be by patient: consecutive five-minute windows from one stay are near-identical, so a window-level split puts the same individual on both sides and inflates AUROC towards 1.0. The abort is correct behaviour.

8.11 CUDA error: invalid configuration argument

A single forward pass over tens of thousands of sequences exceeds CUDA grid limits. Use `predict_batched`, which all current call sites do.

9. Reproducibility notes

Results in the paper come from a random sample of 5,000 stays under seed 42, giving 4,649 patients and 314,301 windows. Reproduce exactly with `--max-stays 5000 --sample-seed 42`.

Model performance depends on the MIMIC-IV version, the itemid mapping, and the outcome definition. **Verify ITEMID_TO_LOINC in `medroad_v3/training/etl.py` against `d_items` and `d_labitems` for your release:** itemids are not stable across versions, and a wrong mapping trains cleanly while meaning nothing.

MIMIC-IV records troponin T where the paper specifies troponin I. These are different assays with different reference ranges. Document the substitution if you report results.

10. Publishing to GitHub

For the repository owner. Skip if you are installing rather than publishing.

10.1 Before the first push

```
python scripts/check_before_push.py
```

It reports credentials, clinical data and model artefacts anywhere in the tree, and lists the placeholders that need filling. It exits non-zero if anything blocking is found. Three decisions it will remind you about:

License. The repository ships MIT. Confirm this is compatible with your institution’s intellectual property policy before publishing; university and company IP terms sometimes constrain the choice.

Repository URL. `pyproject.toml`, `README.md` and `CITATION.cff` all reference `github.com/pboulanger/medroad-v3`. Change them if your account or organisation differs.

ORCID. `CITATION.cff` carries a placeholder. Fill it, so the “Cite this repository” button resolves correctly.

Never commit `.env`, the MIMIC-IV extract, the window matrix, or trained models. `.gitignore` blocks all of these, and the check above verifies it worked. A committed MIMIC-IV CSV is a data use agreement breach, not an inconvenience, and git history makes it hard to undo.

10.2 Create and push

Install git from <https://git-scm.com/downloads> if needed, then:

```
cd medroad-v3
git init
git add .
git status                # read this list before committing
git commit -m "MedROAD V3: real-time FHIR sensor fusion for clinical decision support"
```

Create an empty repository on GitHub, without a README, license or `.gitignore`, since the repository already has all three. Then:

```
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/medroad-v3.git
git push -u origin main
```

With the GitHub CLI the creation and push are one step:

```
gh repo create medroad-v3 --public --source=. --push
```

10.3 After pushing

Continuous integration runs automatically: `.github/workflows/ci.yml` lints and tests on Python 3.11 and 3.12 for every push and pull request. Check the Actions tab after the first push.

Add repository topics so the work is findable: `fhir`, `clinical-decision-support`, `sensor-fusion`, `telemedicine`, `mimic-iv`, `explainable-ai`.

For a citable archive, connect the repository to Zenodo at <https://zenodo.org/account/settings/github/> and create a GitHub release. Zenodo mints a DOI per release, which belongs in the paper's data availability statement and in `CITATION.cff`.

Consider whether to commit the built manual. `docs/MedROAD_V3_Installation.pdf` goes stale whenever `INSTALL.md` changes without a rebuild. Attaching it to releases and adding `docs/*.pdf` to `.gitignore` avoids that.

11. Getting help

Open an issue with your Python version, OS, the command run, and the full traceback. For pipeline problems, `results/retrain_report.json` records which gates passed and is the most useful attachment.